Comparisons

26 April 2026 18:28

We know many comparison operators from maths.
In JavaScript they are written like this:

- Greater/less than: a > b, a < b
- Greater/less than or equals: a >= b, a <= b
- Equals: a == b, Please note the double equality sign == means the equality test, while a single one a = b means an assignment.

- Not equals: In math, the notation is ≠, but in JS its written as **a != b**

Boolean is the result

All comparison operator return a boolean value:

- true -> Means yes, correct or the truth
- false -> Means no wrong or not the truth.

Example:

```
> 2 > 1
true
> 2 == 1
false
> 2 != 1
true
> let result = 5 > 4;
undefined
> typeof(result)
'boolean'
```

String comparison

To see whether a string is greater than other, Javascript uses so called "dictionary" or "lexicographical" order.

In other words, strings are compared letter-by-letter.

For example:

```
> 'Z' > 'A'
true
> 'Glow' > 'Glew'
true
> 'Bee' > 'Bee'
false
> 'Bee' > 'Be'
true
```

The algorithm to compare two string is simple:

1. Compare the first character of both strings.\
2. If the first character from the first string is greater (or less) than the other strings, then the 1st string is greater (or less) then the 2nd. We are done.
3. Otherwise if. Strings first characters are same, compare the second characters the same way.
4. Repeat until the end of either string.
5. If both strings end at the same length, then they are equal. Otherwise the longer string is greater.

Comparison of different types
When comparing values of different types, Javascript convert the values to numbers.

For example:

```
> '01' == 1
true
> '2' > 1
true
> true == 0
false
> true == 1
true
```

Strict equality
A regular equality check **==** has a problem. It cannot differentiate zero from false:

```
> 0 == false
true
> '' == false
true
```

Here this happens because operants of different types are converted to numbers by the equality operator ==. An empty string, just like false, becomes a zero.

What to do if we do like to differentiate zero from false?

A strict equality operator === checks the equality without type conversion.

In other words, if A and B are of different types, then **a === b** immediately returns **false** without an attempt to convert them.

```
> 0 === false;    // false, because the types are different
false
```

There is also a "strict non-equality" operator !== analogous to !=

The strict equality operator is a bit longer to write, but make it obvious what's going on and leave the less room for errors.

Comparison with null and undefined
There's a non-intuitive behavior where null or undefined are compared to other values.

```
> null === undefined
false
```

For math and other comparisons <, >, <=, >=
null/undefined Are converted to numbers: null becomes 0, while undefined becomes NaN.

Strange result:

```
> null > 0
false
> null == 0
```

false
> null >= 0
true

Mathematically, that's strange. The last result states that "**null** is greater than or equal to zero". So in one of the comparison above, it must be true, but they are both false.
The reason is that an equality check == and comparison >, <, >=, <= work differently. Comparison convert null to a number, treating it as zero. That's why 3rd comparison is true and other comparisons were false.

On the other hand, the equality check == for undefined and null is defined such that, without any conversions, they are equal each other and don't equal anything else. That's why 2 null == 0 is false.

An incomparable undefined
The value **undefined** shouldn't be compared to other values.

1. > undefined > 0
   false
2. > undefined < 0
   false
3. > undefined == 0
   false

Why does it dislike zero so much? Always false!
We get the results because:

- Comparison one and two return false because undefined gets converted to Nan and Nan is a special numeric value which returns false for all comparisons.
- The equality check three returns false because undefined only equals null, undefined and no other value.

Avoid problems

Why did we go over these examples? Should we remember these peculiarities all the time? Well, not really. Actually, these tricky things will gradually become familiar overtime, but there's a solid way to avoid problems with them:

- Treat any comparison with undefined or null except the strict equality **"==="** with exceptional care.
- Don't use comparisons >=, >, <, <= With a variable which may be null or undefined unless you are really sure of what you are doing. If a variable can have these values, check for them separately.